Thread-Safe Global Configuration Architect

Centralized configuration management is a cornerstone of enterprise application development. By implementing a thread-safe Singleton pattern, developers can ensure that application settings are loaded exactly once and remain consistent across all execution threads, preventing race conditions and resource duplication in high-concurrency environments.

Implement a `ConfigLoader` class that utilizes the **Singleton** pattern to manage application-wide settings. The architect must provide methods to load key-value pairs from a dictionary, retrieve specific values with optional defaults, and maintain a globally unique state even when multiple instances are requested.

Example 1 Input:

```
LOAD {"api": "v1", "port": 8080} GET api GET port
```

Output:

```
v1 8080
```

Explanation: The configuration is loaded into the singleton instance. Subsequent `GET` requests retrieve the stored values correctly.

Example 2 Input:

```
LOAD {"debug": true} INSTANCE c1 INSTANCE c2 CHECK_IDENTITY c1 c2
```

Output:

```
True
```

Explanation: Despite calling the constructor multiple times, the Singleton pattern ensures that `c1` and `c2` refer to the exact same memory address, sharing the same underlying configuration state.

Function Parameter:

- settings (dict): A mapping of configuration keys to their values.
- key (str): The configuration identifier to retrieve.
- default: The fallback value if the key is not found.

Returns

- Any: The retrieved configuration value or the provided default.

Constraints

- Implement the Singleton pattern using the `__new__` method or a base class.
- The system must be resilient to `None` values and empty dictionaries.
- Configuration keys are case-sensitive.

Input Format for Custom Testing Input from stdin contains commands: `LOAD <json_data>`, `GET <key> <optional_default>`, `INSTANCE <id>`, or `CHECK_IDENTITY <id1> <id2>`.

Sample Case 0 Sample Input

```
LOAD {"env": "prod"} GET env
```

Sample Output

```
prod
```

Explanation The environment setting is successfully loaded and retrieved from the global configuration store.